

Handleiding: Nieuwe Blockly-blokken toevoegen

Introductie

Tijdens Sprint 5 is Blockly uitgebreid met meerdere nieuwe Selenium-blokken, zoals:

- Open Browser
- Input Text
- Click Element
- Wait For Element
- Capture Screenshot
- Close Browser

Om nieuwe Blockly-blokken toe te voegen zijn voornamelijk drie bestanden gebruikt:

1. custom_blocks.js
2. blockly.html
3. code_generator.py

Deze handleiding beschrijft hoe een nieuwe student eenvoudig een extra blok kan toevoegen.

1. Onderzoek naar Blockly

Voordat de implementatie begon is eerst onderzoek gedaan naar de Blockly-documentatie.

Gebruikte documentatie:

https://docs.blockly.com/guides/contribute/samples/block_libraries/

Deze documentatie laat zien hoe Blockly-blokken worden opgebouwd, welke eigenschappen een blok kan bevatten en hoe Blockly nieuwe blokken registreert.

Door deze documentatie te bestuderen werd duidelijk:

- Hoe een blok wordt gedefinieerd.
 - Hoe invoervelden worden toegevoegd.
 - Hoe Blockly blokken groepeert.
 - Hoe een blok een uniek type krijgt.
 - Hoe een blok later kan worden vertaald naar uitvoerbare code.
-

2. Nieuw blok maken in custom_blocks.js

Het bestand custom_blocks.js bevat alle Blockly-blokken van het project.

Een nieuw blok wordt toegevoegd via:

```
Blockly.Blocks['mijn_blok'] = {  
  init: function() {  
    this.appendDummyInput()  
      .appendField("Mijn Blok");  
  
    this.setPreviousStatement(true);  
    this.setNextStatement(true);  
  
    this.setColour(230);  
  }  
};
```

Belangrijke onderdelen:

- De naam (mijn_blok) moet uniek zijn.
- appendField() bepaalt welke tekst zichtbaar is.
- setColour() bepaalt de kleur van het blok.
- setPreviousStatement() en setNextStatement() zorgen ervoor dat blokken gekoppeld kunnen worden.

3. Blok toevoegen aan de Blockly toolbox

Alleen een blok maken is niet voldoende.

Blockly moet ook weten in welke categorie het blok zichtbaar wordt.

Dit gebeurt in:

app/templates/blockly.html

Voorbeeld:

```
<category name="Browser" colour="#4C97FF">  
  <block type="mijn_blok"></block>  
</category>
```

Belangrijk:

- De waarde van type moet exact overeenkomen met de naam uit custom_blocks.js.
- De categorie bepaalt waar gebruikers het blok terugvinden.
- Hier worden ook kleuren van categorieën ingesteld.

Voorbeeld categorieën:

- Browser
- Wachten
- Controles
- Navigatie

4. Blockly vertalen naar Robot Framework

Een Blockly-blok moet uiteindelijk Robot Framework-code genereren.

Hiervoor wordt gebruik gemaakt van:

app/blockly/code_generator.py

In dit bestand staat de BLOCK_MAP.

Voorbeeld:

```
BLOCK_MAP = {
    "open_browser": ("Open Browser", ["URL"]),
    "input_text": ("Input Text", ["FIELD", "TEXT"]),
    "capture_screenshot": ("Capture Page Screenshot", [])
}
```

De BLOCK_MAP koppelt:

- het Blockly-blok
- aan een Robot Framework keyword

Wanneer een gebruiker een Blockly-test uitvoert:

1. Blockly genereert XML.
 2. De XML wordt gelezen door RobotCodeGenerator.
 3. De BLOCK_MAP bepaalt welk Robot Framework keyword gebruikt wordt.
 4. Robot Framework voert de test uit.
-

Samenvatting

Voor het toevoegen van een nieuw blok zijn meestal drie stappen nodig:

Stap 1

Maak het blok aan in:

`custom_blocks.js`

Stap 2

Voeg het blok toe aan een categorie in:

`blockly.html`

Stap 3

Koppel het blok aan een Robot Framework keyword in:

`code_generator.py`

Na deze drie stappen is een nieuw Blockly-blok beschikbaar voor gebruikers en kan het worden gebruikt binnen testscenario's.